

AI原生嵌入式系统编程

综合性实验项目任务书（V2）

项目名称：基于ESP32的AI辅助数字钢琴与AI原生开发工具链

项目周期：2026年7月6日 — 2026年8月9日（共5周）

适用对象：25级实验班

一、项目概述

1.1 项目背景

嵌入式系统是支撑现代社会的核心技术之一。从家用电器、可穿戴设备到工业自动化、智能交通、医疗设备，嵌入式系统无处不在。然而，嵌入式系统开发不仅要求掌握软件编程技能，还需要理解电子硬件、通信接口、时序约束和系统集成等多方面知识。

长期以来，嵌入式系统教学以C语言或MicroPython编程为主，学生通过手动编写程序、烧录验证、反复调试来完成实验。近年来，以Claude Code、Codex、Zcode等为代表的大语言模型和AI编程助手正在改变这一传统工作模式。AI能够理解自然语言指令、生成代码、解释概念、定位缺陷，极大地降低了学习门槛和开发时间。

但是，当前AI编程助手存在一个显著短板：**它们基本无法直接与物理硬件交互**。AI生成一段ESP32的MicroPython程序后，工程师仍需手动上传程序、重启开发板、观察串口输出、收集运行日志、诊断硬件故障，然后往复迭代。代码生成、部署、执行、观测、调试，这些环节至今仍然是断裂的人工操作。AI被限制在**纯软件层面**，而嵌入式开发中最重要的硬件闭环却无法自动化。

本项目旨在引入一种全新的工程范式——**AI原生嵌入式开发**：不仅让AI辅助生成代码，更让AI能够**直接与嵌入式硬件通信**，将代码生成→自动部署→执行观测→日志反馈→AI诊断与修复→再次迭代，整合为一条连续的工程管线。

为此，本项目包含**两个相互关联的工程任务**：

任务	说明
任务一：数字钢琴	利用ESP32开发板实现一台功能完整的数字钢琴（按键输入→音符发声→LED反馈）
任务二：AI原生开发工具链	设计并实现一套工具链，使AI编程助手能直接与ESP32开发板交互（文件传输、程序执行、串口监控、错误报告等），并以数字钢琴为验证基准，演示AI通过工具链驱动嵌入式开发的完整闭环

两个任务相互支撑，不可分割。数字钢琴是**应用载体**，工具链是**工程基础设施**。工具链使得AI能够“触碰”硬件，而数字钢琴则是检验工具链有效性的工程基准。

1.2 硬件平台

本项目的硬件平台是一块ESP32开发板，集成了ESP32-D0WD-V3微控制器及多种板载外设。学生将获得该开发板的PCB版图、原理图和工程物料清单（Engineering BOM）。与以往简化过的实验指导不同，本项目要求学生**直接解读原始工程文档**，从中识别可用硬件资源、确定外设连接关系、设计软件架构。

1.3 为什么两个任务必须一起做

在传统嵌入式开发中，工程师与AI的对话止步于代码生成。AI说“这段代码应该能工作”，工程师去烧录、观察、回来告诉AI“它不能，串口报了这个错”，AI再修改，工程师再去烧录……这本质上还是**人在充当AI与硬件之间的传声筒**。

本项目的核心命题是：**打破这个传声筒模式**。实现方式是让AI编程助手自己掌握与ESP32通信的能力——通过你开发的工具链。届时，AI可以直接：

- 将生成的代码推送到ESP32；
- 远程启动程序；
- 实时监控串口输出；
- 自动捕获运行时错误；
- 根据错误信息自行修复并重新部署。

你的数字钢琴，必须在这个闭环中被开发和验证。**只有当你亲眼看到AI通过你的工具链自主完成一次“发现问题→修改代码→重新部署→验证通过”的迭代，你才算真正完成了这个项目。**

二、教学目标

- 硬件文档解读能力**：能够阅读原理图、PCB版图、物料清单，从中确定外设接口和GPIO分配，为软件设计提供依据。
 - 嵌入式软件设计能力**：能够将工程问题分解为独立的软件模块（硬件抽象、外设驱动、应用逻辑、用户交互、调试支持），形成可维护的系统架构。
 - AI协作开发能力**：能够向AI编程助手清晰描述工程问题，评估生成代码的正确性，通过实验验证AI输出，在AI的辅助下完成高质量的嵌入式软件开发。
 - 开发者工具与基础设施设计能力**：理解AI编程助手的工具调用架构（MCP、插件系统、扩展机制），能够设计并实现嵌入式硬件交互工具，使之被AI编程助手以程序化方式调用。这是本课程区别于传统嵌入式课程的核心教学目标之一。
 - 系统调试与问题解决能力**：能够采用结构化的调试方法（观察→假设→实验→结论），定位并解决嵌入式系统中的软硬件问题。
 - 技术沟通与工程文档能力**：能够通过GitHub仓库管理代码、通过周进度视频汇报进展、撰写规范的技术报告，形成专业的技术沟通能力。
 - 持续学习与批判性思维**：能够在AI辅助环境中保持独立思考，养成验证、评估、反思的习惯。理解AI编程助手的内部架构，从而更有效地与之协作。
-

三、硬件平台分析

3.1 分析要求

在编写任何代码之前，学生必须首先理解硬件平台。请仔细阅读以下三份工程文档：

文档	用途
原理图 (Schematic)	描述各电子元器件的逻辑连接关系，确定ESP32与各外设的电气接口
PCB版图 (PCB Layout)	展示元器件的物理布局、连接器位置、板级组织
工程物料清单 (BOM)	列出板上所有元器件的型号、规格和制造商

3.2 分析任务

- 识别板上的**主要功能模块**（微控制器、USB接口、电源电路、按键、LED、蜂鸣器、惯性测量单元等）。
- 确定各外设与ESP32的**GPIO对应关系和通信接口**。
- 编制一份**硬件资源分配表**，列出数字钢琴所需各硬件资源的软件接口。
- 明确哪些板载资源在数字钢琴项目中不使用，并简要说明原因。

3.3 注意事项

- 硬件文档是开发的**权威参考**，不可凭臆测做出假设。
- 在整个项目过程中，应随时回查原理图和PCB版图。
- AI编程助手无法自动获知硬件连接信息，**工程师必须提供准确的硬件上下文**。这也是工具链项目要解决的核心问题之一：如何让AI能够获取硬件状态信息。

四、开发环境

4.1 基本环境要求

组件	要求
开发语言	MicroPython + Python（工具链开发语言）
目标硬件	ESP32-D0WD-V3 开发板
代码编辑器	不限（推荐VS Code）
烧录工具	esptool.py + MicroPython固件
串口工具	用于调试输出和REPL交互
版本控制	Git + GitHub（公共仓库）

4.2 AI编程助手的选用

本项目要求学生在开发过程中持续使用AI编程助手。推荐选项如下：

选项	说明	工具链实现建议
Claude Code	功能成熟，支持MCP扩展	实现MCP服务器
Codex	OpenAI生态	编写Codex兼容插件
Zcode	开源环境	编写Zcode扩展
OpenCode + DeepSeek API	开源 + 低成本	编写OpenCode工具扩展

工具链任务必须与你选用的AI编程助手绑定。你需要在所选平台上实现硬件交互扩展，使AI能够调用你的工具与ESP32通信。

4.3 研读开源参考项目

在正式开始数字钢琴开发之前，**必须**阅读ShareAI实验室的开源教育项目"mini-claude-code"。该项目以简化方式实现了AI编程助手的核心架构，是理解工具链设计的入门钥匙。请着重理解：

- 用户指令如何被解析为工程任务；
- 工具调用（Tool Invocation）的完整流程；
- 项目文件如何被管理和组织；
- 大语言模型如何与工程环境交互；
- 如何向现有框架添加自定义工具。

不要求复现mini-claude-code。但你对工具调用机制的理解深度，将直接决定工具链项目能否顺利完成。

五、项目功能要求

5.1 任务一：数字钢琴（基本要求）

- 按键输入**：至少使用2个板载按键（KEY1、KEY2）作为钢琴键输入，按下按键时触发音符播放。
- 音符生成**：通过板载蜂鸣器（MLT-5020）以PWM方式产生不同频率的方波，输出至少7个音阶（do-re-mi-fa-sol-la-si）。
- 视觉反馈**：按下按键时，板载LED应提供即时视觉反馈。
- 系统稳定性**：上电后自动进入工作状态，持续稳定响应多次按键操作。
- 软件模块化**：代码按功能模块化组织，不允许单文件堆砌。
- 扩展功能**（至少选做一项）：八度切换、旋律录制与回放、启动动画、体感控制（MPU6050）等。

5.2 任务二：AI原生开发工具链（基本要求）

工具链须以你选用的AI编程助手为载体（MCP服务器、插件或扩展），提供以下**基本工程能力**：

序号	能力	描述
1	文件传输	AI能将MicroPython源文件推送到ESP32开发板
2	程序执行	AI能远程触发用户程序的运行和停止
3	微控制器复位	AI能通过软件命令重启ESP32
4	串口监控	AI能实时捕获并读取ESP32的串口输出
5	运行日志检索	AI能收集程序执行日志
6	错误报告	AI能自动检测MicroPython运行时异常

进阶功能（选做，至少选做一项）：

- 固件自动烧录
- 远程文件管理（目录浏览、文件删除等）
- GPIO状态在线查询
- 硬件信息自动采集
- 自动化回归测试
- 执行性能分析

5.3 闭环验证（核心交付）

这是整个项目最重要的验收环节。你必须录制并演示以下完整闭环：

- AI编程助手生成/修改数字钢琴代码
↓
- AI调用你的工具链 → 自动上传代码至ESP32
↓
- AI调用你的工具链 → 远程启动程序
↓
- AI调用你的工具链 → 实时监控串口输出
↓
- 若出现异常，AI自动捕获错误信息
↓

6. AI根据错误信息自主修复代码

↓

7. AI重新部署修复后的代码（回到步骤2）

↓

8. 循环直至功能正常运行

你必须展示至少一次完整的**AI自主迭代闭环**（AI发现问题→修改代码→自动部署→验证通过）。如果只有代码生成而无硬件交互自动闭环，工具链任务视为未完成。

六、项目里程碑与时间安排

本项目分为五个阶段，每一阶段同时推进数字钢琴和工具链的开发，两者交替进行、相互促进。

阶段	时间区间	截止日期	主题	核心产出
第一周	7月6日 — 7月12日	7月12日 (周日)	硬件认知与AI协作入门	硬件分析文档 + 外设测试代码 + mini-claude-code阅读笔记
第二周	7月13日 — 7月19日	7月19日 (周日)	数字钢琴核心实现 + 工具链架构设计	可运行的数字钢琴v1 + 工具链技术方案文档
第三周	7月20日 — 7月26日	7月26日 (周日)	工具链基础功能开发	工具链基本功能可用（文件传输、执行、串口监控、错误报告）
第四周	7月27日 — 8月2日	8月2日 (周日)	工具链集成与闭环验证	AI通过工具链自主操控ESP32的完整演示
第五周	8月3日 — 8月9日	8月9日 (周日)	系统完善与项目收尾	最终完整演示 + 最终技术报告 + GitHub仓库整理

各阶段详细说明：

第一周：硬件认知与AI协作入门（7月6日 — 7月12日）

数字钢琴方面：

- 仔细阅读原理图、PCB版图和BOM，编制硬件资源分析文档。
- 搭建MicroPython开发环境，烧录固件到ESP32。
- 在AI辅助下，编写并运行测试程序，逐个验证板载外设（按键、LED、蜂鸣器）。

工具链方面：

- 研读mini-claude-code源码，理解工具调用架构。
- 明确你将使用哪个AI编程助手（Claude Code / Codex / Zcode / OpenCode），研究其工具扩展机制（MCP协议 / 插件API / 扩展接口）。
- 初步确定工具链的实现方案和技术选型。

第一周截止：7月12日（周日）23:59前 提交进度视频至Bilibili并分享链接至QQ群。

第二周：数字钢琴核心实现 + 工具链架构设计（7月13日 — 7月19日）

数字钢琴方面：

- 在AI辅助下实现数字钢琴核心功能（按键输入→音符生成→LED反馈）。
- 确保代码模块化组织。
- 在硬件上完成初步功能验证。

工具链方面：

- 完成工具链的详细架构设计文档。
- 确定各工具的接口定义（文件传输工具的输入输出格式、串口监控的数据格式、错误报告的协议等）。
- 搭建工具链开发框架，实现第一个可工作的工具（建议从串口监控或文件传输开始）。

第二周截止：7月19日（周日）23:59前 提交进度视频至Bilibili并分享链接至QQ群。

第三周：工具链基础功能开发（7月20日 — 7月26日）

工具链方面（本周重心）：

- 实现全部6项基本工程能力（文件传输、程序执行、复位、串口监控、日志检索、错误报告）。
- 每个工具独立测试通过。
- 将工具链集成到AI编程助手中，验证AI能够成功调用每个工具。

数字钢琴方面：

- 根据工具链开发过程中对硬件的深入理解，优化和完善数字钢琴代码。
- 开始尝试通过工具链（而非手动）来部署和测试数字钢琴。

第三周截止：7月26日（周日）23:59前 提交进度视频至Bilibili并分享链接至QQ群。

第四周：工具链集成与闭环验证（7月27日 — 8月2日）

闭环验证（本周重心）：

- 实现并录制完整的AI自主迭代闭环：AI发现钢琴代码中的问题→自动修改代码→通过工具链自动部署到ESP32→监控运行结果→确认修复。
- 这是整个项目最关键的工程证据，务必充分准备。

数字钢琴方面：

- 完成选做的扩展功能。
- 通过AI+工具链的闭环进行最终的代码优化和Bug修复。

第四周截止：8月2日（周日）23:59前 提交进度视频至Bilibili并分享链接至QQ群。

第五周：系统完善与项目收尾（8月3日 — 8月9日）

- 录制最终完整演示视频（同时展示数字钢琴功能与AI闭环开发流程）。
- 整理GitHub仓库，确保代码、文档齐全且组织规范。
- 撰写最终技术报告（涵盖两个任务的完整工程过程）。
- 将所有材料提交完毕。

第五周截止：8月9日（周日）23:59前 完成所有材料的提交。

重要提示：以上截止时间均为北京时间（GMT+8）23:59。逾期提交将按考核标准扣分。

七、每周进度视频要求

7.1 发布方式

每周进度视频须发布在 **Bilibili（哔哩哔哩）** 平台，并将视频链接分享至 **实验班QQ群**。视频**不需要**上传至GitHub仓库。

7.2 内容要求

每周视频应包含以下内容：

- 本周目标回顾**：说明本周数字钢琴和工具链各自的工程目标。
- 进度演示**：在**真实硬件**上演示当前系统状态（严禁使用仿真或静态截图代替）。工具链部分需展示AI调用工具的屏幕录制。
- 技术决策说明**：解释本周做出的重要技术决策及其理由。
- AI协作实录**：说明AI编程助手本周具体协助了哪些工作，并讨论AI输出中不完善或需要修改的情况。
- 问题与反思**：如实汇报未解决的技术难题及尝试过的解决途径。
- 下周计划**：明确下一阶段两个任务的工程目标。

7.3 技术要求

- 视频时长建议 **8—15分钟**（因包含两个任务的汇报，可适当延长）。
- 演示必须在**物理硬件**上进行，不得仅展示代码或截图。
- 可以使用屏幕录制、硬件实拍、图文解说等多种形式。
- 如涉及团队分工，请说明每个成员在本周的贡献。

八、最终提交要求

8.1 提交清单

序号	提交项	提交方式	截止时间
1	最终演示视频	发布至Bilibili，链接分享至QQ群	8月9日
2	数字钢琴完整源代码	GitHub公共仓库	8月9日
3	工具链完整源代码	GitHub公共仓库（可同仓库）	8月9日
4	最终技术报告（PDF格式）	发送至教师邮箱	8月9日
5	全部5次周进度视频	Bilibili（链接已分享至QQ群）	每周日

8.2 最终演示要求

- **数字钢琴部分：**在物理硬件上演示全部功能，包括基本功能和扩展功能。
- **工具链部分：**演示AI通过工具链与ESP32交互的完整闭环。必须展示至少一次AI自主发现问题→修改代码→自动部署→验证通过的迭代过程。
- 能够清晰解释两个任务的软件架构和各模块职责。
- 能够回答关于硬件接口、工具调用机制和AI协作策略的问题。

8.3 GitHub仓库要求

- 仓库应反映从第一周到第五周的**渐进式开发过程**。
- 提交记录应频繁且有意义的提交消息。
- **数字钢琴代码和工具链代码**应分目录清晰存放。
- 推荐目录结构：

```
ESP32-AI-Piano/
├── README.md           # 项目简介、硬件平台、运行方法
├── LICENSE
├── .gitignore
├── firmware/          # 数字钢琴 MicroPython 源代码
│   ├── main.py
│   ├── buzzer.py
│   ├── buttons.py
│   ├── leds.py
│   └── piano.py
├── toolchain/         # AI原生开发工具链源代码
│   ├── mcp_server.py  # 或 plugin.py / extension.py
│   ├── tools/
│   │   ├── file_transfer.py
│   │   ├── serial_monitor.py
│   │   ├── executor.py
│   │   └── error_handler.py
│   └── README.md      # 工具链安装与使用说明
├── hardware/          # 硬件工程文档
│   ├── schematic.pdf
│   ├── pcb.pdf
│   └── bom_analysis.md
```

```
|— docs/          # 补充技术文档
|— tests/         # 外设测试程序
|— images/
|— report/        # 最终报告源文件
```

8.4 最终报告提交

最终报告须以PDF格式发送至教师邮箱（邮箱地址：`349744305@qq.com` 务必在邮件内和报告内，均注明每组的所有成员名字）。

邮件主题格式：`[数字钢琴与工具链项目] 组长姓名_学号_最终报告`

九、最终技术报告

9.1 报告定位

最终报告是项目的核心文档，应系统阐述两个任务的完整工程过程。数字钢琴与工具链相互关联，报告中应清晰展示两者之间的关系。

9.2 报告结构建议

- 引言（~1页）
 - 项目背景与目标
 - 两个任务的关系概述
- 硬件平台分析（~2页）
 - 硬件架构与功能模块
 - GPIO分配与接口分析
 - 数字钢琴实现中的硬件资源选用及理由
- 任务一：数字钢琴设计与实现（~4页）
 - 软件架构与模块划分
 - 各模块功能说明
 - 关键实现技术（可附简短代码片段）
 - 扩展功能说明
- 任务二：AI原生开发工具链设计与实现（~5页）
 - 工具链架构设计（附架构图）
 - 各工具模块的详细实现（文件传输、串口监控、程序执行、错误报告等）
 - 与AI编程助手的集成方式（MCP / 插件 / 扩展机制说明）
 - 关键技术的实现细节
- 闭环验证与分析（~2页）
 - AI通过工具链自主迭代的完整过程记录
 - 闭环验证中遇到的问题及解决
 - 工具链对开发效率的实际影响分析
- AI协作反思（~2页）
 - 两个任务中AI的贡献与局限（附具体案例）
 - AI在代码生成 vs 工具链开发中的不同表现
 - 对AI辅助嵌入式开发的思考
- 总结与展望（~1页）
 - 主要成果
 - 经验教训
 - 未来改进方向

9.3 报告要求

- 篇幅：正文**15—20页**（不含附录，因包含两个任务的完整描述）。
- 语言：**中文**，技术术语可使用英文原名。

- 图表：所有图表应有编号和标题，在正文中有引用。
 - 引用：如参考了外部资料，须列出参考文献。
 - 格式：A4纸，宋体小四号正文，1.5倍行距。
-

十、考核标准

考核项	权重	评价要点
周进度视频（5次 × 4%）	20%	内容完整性、硬件演示真实性、两个任务的进展汇报、按时提交
GitHub仓库	15%	仓库结构、提交历史连续性、数字钢琴代码质量、工具链代码质量
数字钢琴实现与演示	25%	基本功能完整、系统稳定性、代码架构、扩展功能
AI原生工具链实现与演示	25%	6项基本能力完成度、AI集成成功验证、闭环迭代演示、进阶功能
最终技术报告	10%	两个任务的完整性、技术分析深度、AI协作反思、写作规范性
综合工程素养	5%	进度管理、批判性思维、学术诚信

特别注意：工具链任务的**闭环验证演示**（AI通过工具链自主完成一次"发现错误→修改代码→部署→验证"迭代）是工具链考核的核心。不具备该闭环能力的工具链，该项得分不超过该分项的40%。

十一、学术诚信与AI使用规范

11.1 AI使用原则

1. **工程师负最终责任**：每一行提交的代码都必须被学生完全理解。
2. **硬件验证是唯一标准**：AI生成的代码必须在实际硬件上运行验证通过后方可采纳。
3. **如实记录AI贡献**：在周视频和报告中如实说明AI的贡献和不足。

11.2 关于工具链的特别声明

工具链是工程师设计的**工程基础设施**，它与AI生成的应用代码有本质区别：

- AI可以辅助你编写工具链的代码，但它**不能替你设计工具链架构**。
- 你必须是工具链**总体架构的决策者**，理解每一个工具为什么需要、接口为什么这样定义、错误如何处理。
- 工具链的闭环验证必须反映你**设计工具链时的工程意图**，而非AI漫无目的的自动行为。

11.3 学术诚信底线

- 禁止抄袭其他团队的代码或报告。
- 禁止伪造闭环验证的演示（例如预录脚本冒充AI自主交互）。
- 允许参考开源代码和在线教程，但必须注明出处。
- 如果教师问“你的工具链中串口监控模块为什么这样设计”而你不能回答，那么无论演示多么流畅，该部分都不会被认可。

AI可以帮你写代码，但AI不能替你做工程架构决策。

本项目培训的核心能力是：**你知道该让AI做什么，你知道AI做出来的东西对不对，你知道如何将AI的能力通过工程基础设施延伸到物理世界。** 如果你做完这个项目后只能说“AI帮我写了代码”，而没有自己的架构理解和工程判断，那么项目目标没有达成。

附录A：开发板硬件资源清单

A.1 工程物料清单

功能模块	主要器件	功能说明	数字钢琴是否必需
微控制器	ESP32-D0WD-V3	执行MicroPython固件，控制全部外设	是
Flash存储	ZB25VQ32（32Mbit SPI Flash）	存储固件和用户程序	是
时钟电路	40MHz晶振	提供系统时钟	是
USB编程接口	CP2102N + Micro-USB	程序下载和串口通信	是
电源电路	AMS1085 × 2（5V + 3.3V）	电压转换与稳压	是
用户按键	BOOT、RESET、KEY1、KEY2	用户输入	是
LED指示灯	LED1（红，电源）、LED2（绿）、LED3（红）	状态指示	是
蜂鸣器	MLT-5020 压电蜂鸣器	音符发声	是
惯性测量单元	MPU6050（六轴IMU）	运动姿态检测（I²C）	否（扩展）
电机驱动	MX620B（H桥）	电机控制	否（扩展）
GPIO排针	多组排针引出	外部扩展	否（扩展）

A.2 GPIO关键对应关系

外设	GPIO	备注
按键KEY1	GPIO34	输入，内部上拉
按键KEY2	GPIO35	输入，内部上拉
绿色LED（LED2）	GPIO32	输出，低电平有效
红色LED（LED3）	GPIO33	输出，低电平有效
蜂鸣器（B1）	GPIO25	PWM输出，经晶体管驱动
BOOT按键	GPIO0	输入
MPU6050 SDA	GPIO16	I²C数据线
MPU6050 SCL	GPIO17	I²C时钟线

以原理图为最终依据。

附录B：工具链技术参考

B.1 可选架构方案对比

架构	AI编程助手	实现方式	难度	推荐度
MCP服务器	Claude Code / 支持MCP的客户端	实现Model Context Protocol服务器，暴露ESP32工具	★★★★	★★★★★
Codex插件	Codex	编写Codex兼容的工具插件	★★★★	★★★★
Zcode扩展	Zcode	编写Zcode扩展模块	★★★★	★★★★
OpenCode扩展	OpenCode	增加OpenCode自定义工具	★★★	★★★
自定义CLI + AI Agent	任意	编写命令行工具集，由AI Agent通过Shell调用	★	★★

B.2 MCP服务器实现要点（推荐方案）

Model Context Protocol (MCP) 是Anthropic推出的开放协议，允许AI编程助手通过标准化接口调用外部工具。实现要点：

- 1. **工具注册**：向MCP客户端注册你的ESP32工具列表及参数定义。
- 2. **串口通信**：使用Python的 `pyserial` 库与ESP32的CP2102N串口通信。
- 3. **文件传输**：通过串口REPL模式发送 `ampy` 风格的命令或直接使用MicroPython的 `raw REPL` 协议。
- 4. **错误解析**：解析MicroPython的Traceback输出，提取文件名、行号和错误类型。
- 5. **异步监控**：使用独立线程持续监听串口，捕获ESP32输出并转发给AI。

B.3 推荐技术栈

用途	推荐技术
工具链开发语言	Python 3.10+
串口通信	pyserial
MCP框架	mcp (Python SDK)
文件传输协议	MicroPython raw REPL / ampy协议
异步处理	asyncio / threading
测试	pytest

B.4 关键参考资料

- mini-claude-code (ShareAI) : <https://github.com/ShareAI-Lab/mini-claude-code>
- Model Context Protocol 规范: <https://modelcontextprotocol.io>
- MicroPython REPL文档: <https://docs.micropython.org/en/latest/reference/repl.html>
- ampy工具源码: <https://github.com/scientifichackers/ampy>

附录C：AI辅助开发实战指南

C.1 基本原则

原则一：先定义问题，再请求代码。 AI在有清晰目标、充分上下文和明确约束时表现最佳。

原则二：先分析硬件，再编写软件。 每次实现前，先查阅原理图和PCB版图。GPIO编号、通信接口、电平要求等硬件信息必须准确提供给AI。

原则三：增量开发，逐模块验证。 将数字钢琴拆分为独立模块，将工具链拆分为独立工具，先分别开发和验证，再集成。小步快跑降低调试难度。

原则四：所有代码必须上硬件验证。 AI生成的代码逻辑再完美，也要在ESP32上实际运行测试。

原则五：调试是工程调查，而非试错。 先收集错误信息，再向AI求助，逐步排除，而不是同时修改多个变量。

原则六：保持与AI的持续对话。 不要仅把AI当代码生成器。讨论架构方案、请教硬件接口、对比实现策略——AI是你整个项目的技术协作者。

原则七：边做边记，勿靠回忆。 重要的观察、设计决策、调试策略应及时记录。

原则八：批判性地评估AI输出。 AI可能生成不完整、低效或与硬件不匹配的代码。始终将AI输出与硬件文档和实验结果比对。

原则九：工程就是迭代。 硬件分析、软件实现、调试、实验和文档之间持续互相影响。修改不是失败，而是工程常态。

原则十：工程师负责，AI永远辅助。 你负责理解硬件、设计架构、验证软件、评估结果。本项目的目的不是测试AI能否替代嵌入式工程师，而是让你学会借助AI成为更强的工程师。

C.2 推荐AI对话模板

【上下文】 我使用的是ESP32-D0WD-V3开发板，板载蜂鸣器MLT-5020
连接在GPIO25，通过NPN晶体管（MMSS8050）驱动，高电平使能。
固件是MicroPython。

【问题】 请帮我编写一个MicroPython模块，实现以下功能：

- 使用PWM驱动GPIO25上的蜂鸣器；
- 能够播放do-re-mi-fa-sol-la-si七个音阶；
- 每个音符持续0.3秒；
- 提供play_note(freq, duration)和play_melody(notes)两个接口。

【约束】 使用machine.PWM，不依赖第三方库。

关键要素：硬件上下文 + 明确需求 + 具体约束。

注意：Deepseek API不是多模态的模型，它可能无法正确理解两个PCB相关的PDF文件。建议用百度AI Studio的PaddleOCR先把PDF转成MD格式，或者使用GLM大模型的API替代Deepseek API。

附录D：常见问题解答（FAQ）

Q1：两个任务都很难，时间够吗？

5周时间确实紧凑，但两个任务相互促进，不是简单相加。数字钢琴是工具链的验证载体，工具链开发中你会对硬件有更深入的理解，反过来优化数字钢琴。关键在于**不要将两个任务视为独立项目**——它们是同一枚硬币的两面。

Q2：工具链要多复杂才算合格？

6项基本能力 + 至少1次完整的AI自主迭代闭环。不需要UI界面，不需要完美的错误处理，不需要兼容所有操作系统——能用、能演示闭环，即为合格。

Q3：我不会写MCP服务器/插件怎么办？

这正是为什么要先研读mini-claude-code。5周内从零学会这些是可行的，而且AI编程助手会在这个过程中协助你设计工具链本身——这是一个**元循环**（Meta-loop）：你用AI辅助开发工具链，工具链又让AI能直接操控硬件。这个元循环本身就是本课程最深刻的学习体验。

Q4：团队如何分工？

每个成员可以只负责某一方面，但是都必须理解两个任务的完整架构。每周的视频可以由不同组员轮流录制。最终报告只需要每一组写一份即可。

Q5：可以用现成的ESP32工具库吗？

可以，但必须理解其工作原理。工具链的核心价值在于“让AI能调用”，而非“你自己从零实现所有通信协议”。可以使用pyserial、esptool等成熟库，但AI调用工具的接口必须是你自己设计的。